

Voice-Driven Intelligent Desktop Workflow Agent Using Natural Language Understanding and Personalized Workflow Learning

Siva Sanjay S

Q Batch

Dept. of Computer Science and Engineering

Balakrishnan R

Q Batch

Dept. of Computer Science and Engineering

Abstract

Modern desktop computing environments often require users to manually navigate applications and execute repetitive commands to establish task-specific workflows. Although contemporary voice assistants offer automation capabilities, they predominantly operate on a rigid, direct-command basis and lack a contextual understanding of complex user intentions or personalized desktop habits. This paper proposes a Voice-Driven Intelligent Desktop Workflow Agent designed to bridge the gap between natural language understanding and system-level desktop automation. Unlike traditional systems, the proposed architecture decouples cognitive reasoning from physical execution by utilizing a Large Language Model (LLM) strictly as an intent-classification and reasoning component rather than a direct execution engine. The system operates on a modular pipeline comprising voice processing, intent recognition, contextual workflow recommendation, user confirmation, and execution. Furthermore, a continuous preference learning module autonomously updates a local knowledge base to refine future recommendations based on user feedback and temporal patterns. A working prototype has been implemented and made available as open-source software; this paper reports a preliminary result analysis from pilot usage of the prototype alongside a comparative discussion against existing desktop-automation paradigms in the literature.

I. INTRODUCTION

A. Problem Statement

Modern desktop automation tools require users to execute predefined commands or manually launch multiple applications before beginning a task. Although voice assistants exist, they primarily execute direct commands and lack an understanding of the user's workflow, intent, and personalized desktop habits. Consequently, there is a critical need for an intelligent agent capable of inferring latent user goals from natural language and autonomously preparing the necessary desktop environment.

B. Key Contribution

This work proposes a Voice-Driven Intelligent Desktop Workflow Agent that bridges natural language understanding and desktop automation. The key contributions include:

- A modular AI-agent architecture that treats the Large Language Model strictly as an intent-classification and reasoning component, rather than a direct execution engine.
- An intelligent pipeline that integrates speech recognition, intent understanding, workflow planning, user confirmation, and execution.
- A continuous preference learning module that updates a local knowledge base based on user feedback.
- A working, open-source reference implementation, together with a preliminary result analysis and comparative discussion (Section III) grounded in pilot usage data collected from the prototype's local knowledge base.

II. PROPOSED METHODOLOGY

The proposed system adopts a multi-tier modular architecture that replaces the traditional "Command → Execute" pipeline with a "Listen → Understand → Reason → Recommend → Confirm → Execute → Learn" lifecycle, comprising eight modules: Voice Processing, Natural Language Understanding, Intent Recognition (LLM-based), Workflow Recommendation, User Confirmation, Workflow Execution, Preference Learning, and a persistent local Knowledge Base (SQLite).

A. Reference Implementation

A reference implementation of the proposed architecture has been developed in Python and is released as open-source software at:

<https://github.com/sivasanjaycse/local-agent>

The current prototype implements the full reasoning-to-execution pipeline (intent classification → validation → workflow construction → user confirmation → execution → logging) for four intent classes, and persists every interaction to a local SQLite knowledge base, which forms the basis of the result analysis in Section III.

III. RESULTS AND ANALYSIS

A. Evaluation Setup

Results in this section are drawn directly from the prototype’s local knowledge base, populated during pilot usage of the running system. Each user utterance is logged together with its raw transcript, the LLM-normalized text, the classified intent, the generated workflow (as structured JSON), whether the user accepted the recommendation, whether it was subsequently executed, and any OS-level execution error encountered. This provides a small but authentic, end-to-end trace of the agent operating exactly as designed, rather than a simulated benchmark. Because the pilot dataset is small (six logged interactions during early testing), the findings below should be read as preliminary observations that motivate the larger-scale evaluation proposed in future work, not as statistically robust performance claims.

B. Quantitative Observations

Two of the metrics defined for the system, Workflow Acceptance Rate (WAR) and Execution Reliability (ER), could be computed directly from logged pilot data, summarized in Table I.

TABLE I
PRELIMINARY METRICS FROM PILOT USAGE

Metric	Numerator	Denominator	Rate
Workflow Acceptance Rate (WAR)	3 accepted	6 recommended	50.0%
Execution Reliability (ER)	2 successful	3 executed	66.7%

Table II summarizes the six logged pilot sessions, spanning four of the system’s intent classes.

TABLE II
SUMMARY OF LOGGED PILOT SESSIONS

#	Classified Intent	Accepted	Outcome
1	CODING	Yes	Executed successfully
2	CODING	No	Rejected
3	SYSTEM	No	Rejected
4	PRODUCTIVITY	Yes	Executed successfully
5	PRODUCTIVITY	No	Rejected
6	COMMUNICATION	Yes	Executed, OS error

C. Observations

Intent classification proved robust to disfluent and noisy transcripts. Because Module 3 delegates classification to an LLM operating on normalized natural language rather than fixed keywords, informal and imperfectly transcribed utterances were still mapped to the correct intent class. In one logged case, the speech-to-text stage rendered “vibe coding” as “*wipe coding*”; the Intent Recognition Module nonetheless correctly classified the utterance as CODING and correctly inferred the colloquial meaning in its reasoning trace. This is consistent with the architectural premise in Section III that a reasoning-based, LLM-driven classifier generalizes better than brittle keyword matching (cf. [1], [2]).

Rejections were concentrated in over-broad or under-specified workflows. Session 2 (Table II) is illustrative: a short, informal “let’s do some coding” request caused the Workflow Recommendation Module to propose a six-step workflow (VS Code plus five auxiliary websites, including unrelated entertainment and chat tools), which the user rejected. By contrast, the single-step CODING recommendation in Session 1 was accepted immediately. This suggests that, in the current prototype, recommendation scope is not yet well-calibrated to utterance specificity, and that the Preference Learning Module (Module 7) will need explicit signal from rejected multi-step workflows to learn to prefer narrower recommendations, rather than only reinforcing accepted ones.

The one execution failure was infrastructural, not cognitive. In Session 6, the agent correctly classified intent (COMMUNICATION), correctly built a two-step workflow (open browser, open GitHub), and the user accepted it, yet execution failed with an OS-level error (WinError 2: application path not found). This failure is attributable to the Workflow Execution Engine’s reliance on a fixed application path rather than to any error in reasoning upstream, and points to a concrete engineering gap: application launch targets should be resolved dynamically (e.g., via the OS registry or PATH lookup) rather than hardcoded, to raise Execution Reliability independently of intent-classification quality.

The Human-in-the-Loop gate behaved as designed. Across all six sessions, the User Confirmation Module correctly blocked execution for every rejected recommendation (Sessions 2, 3, 5) and only proceeded to execution after explicit acceptance, empirically validating the security design goal stated in Section I-B that the agent should never modify system state without confirmation.

D. Comparative Analysis

Positioned against the reviewed literature, the prototype's behavior reflects the trade-offs the survey anticipated rather than contradicting them. Compared with rule-based offline assistants such as RAY [3], which rely on deterministic intent detection, the present system trades some of that determinism for semantic flexibility: it correctly handled a paraphrased, ASR-corrupted command that a fixed keyword ruleset would likely have missed, supporting the survey finding that hybrid, LLM-driven semantic reasoning outperforms purely rule-based or vision-only automation [2]. At the same time, the 50% acceptance rate observed here is markedly lower than what a well-tuned recommendation system would be expected to achieve, and the over-broad workflow in Session 2 illustrates precisely the risk that motivates memory-augmented and search-based planners such as DMA-MCTS [5] and small-scale planner-evaluator approaches such as PlanEval [6]: without a mechanism to constrain recommendation scope to what the utterance actually warrants, an LLM-driven planner tends toward over-generation. Similarly, the single execution failure observed here is the class of fragility that structured, failure-aware execution layers are designed to address [4], reinforcing that Module 6 (Workflow Execution Engine) is the component most in need of the environment-aware robustness those works describe, rather than the NLU or reasoning stages. Overall, the pilot data supports the paper's central architectural claim, decoupling reasoning from execution improves semantic robustness, while also empirically confirming that the largest remaining gaps are exactly where the literature predicts: recommendation-scope calibration and execution-layer robustness, not intent understanding.

E. Limitations of the Present Analysis

The pilot dataset analyzed here (six interactions, one user, one session block) is too small to support statistically significant claims about WAR, ER, or per-class intent accuracy, and no ground-truth-labeled corpus was yet available to compute Word Error Rate or a true Intent Classification F1-score, consistent with the corpus limitation already noted in Section III-B. These results should therefore be read as a qualitative proof-of-concept of the full pipeline operating end-to-end, motivating the larger-scale, labeled-corpus evaluation proposed as future work.

IV. CONCLUSION

This paper presented a Voice-Driven Intelligent Desktop Workflow Agent that decouples LLM-based reasoning from desktop-level execution within a confirm-before-execute lifecycle. A working reference implementation is available at <https://github.com/sivasanjanycse/local-agent>. Preliminary result analysis on pilot usage data shows that the LLM-based Intent Recognition Module is robust to noisy, colloquial speech, that the confirmation gate reliably prevents unconfirmed system changes, and that the two principal open issues, over-broad workflow recommendations and brittle application-path resolution at execution time, are localized to the Workflow Recommendation and Execution modules rather than to the core reasoning pipeline, giving a clear, evidence-based direction for future refinement.

REFERENCES

- [1] B. Fu, "A Lightweight Semantic-Syntactic Fusion Model for Intent Recognition (LSF-IR) and its Experimental Study," 2026 5th International Conference on Electronic Information Engineering, Big Data and Computer Technology (EIBDCT), Haikou, China, 2026, pp. 374-379, doi: 10.1109/EIBDCT69742.2026.11566754.
- [2] M. C. Shishira, A. Archana, M. Hariprakash, K. Muhammed Niyas and M. Sreelakshmi, "Advancements in GUI Automation: A Survey on Vision, Multimodal Learning, and Hybrid Frameworks," 2026 IEEE International Students' Conference on Electrical, Electronics and Computer Science (SCECS), Bhopal, India, 2026, pp. 1-6, doi: 10.1109/SCECS68810.2026.11429825.
- [3] T. B. Indresh, A. M. Shanbhag, S. K. R. D. Yadav and M. EC, "Ray - An Offline Multimodal Desktop Assistant with Integrated Voice and Gesture Recognition," 2026 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE), Bangalore, India, 2026, pp. 1-6, doi: 10.1109/IITCEE67948.2026.11393991.
- [4] H. Chong, J. Lee and H. Ahn, "Robust Task Planning via Failure Detection Using Scene Graph From Multi-View Images," in IEEE Robotics and Automation Letters, vol. 11, no. 2, pp. 1986-1993, Feb. 2026, doi: 10.1109/LRA.2025.3645659.
- [5] J. Wang, Q. Wang, M. Li, T. Li and Y. Xu, "DMA-MCTS: Dynamic Memory-Augmented Monte-Carlo Tree Search for LLM Task Planning," 2025 IEEE International Conference on Systems, Man, and Cybernetics (SMC), Vienna, Austria, 2025, pp. 6377-6382, doi: 10.1109/SMC58881.2025.11343701.
- [6] W. Li, J. Wang, L. -C. Yu and X. Zhang, "PlanEval: Augmenting Large Language Models With a Small-Scale Planner-Evaluator for Knowledge-Intensive Tasks," in IEEE Transactions on Audio, Speech and Language Processing, vol. 34, pp. 1952-1965, 2026, doi: 10.1109/TASLPRO.2026.3677587.